SPE Final Project

FraudGuard: An MLOps-Based Fraud Detection Platform

MT2025064 Kautilya Singh
MS2025021 Vishesh Tamrakar

May 13, 2026

GitHub Repository:
`https://github.com/Vishesh-tamrakar/Fraudguard-devops-main`

Contents

1 System Overview

FraudGuard is an advanced, production-grade MLOps platform designed to provide real-time fraud detection for financial transactions. By integrating a custom-trained machine learning model within a highly scalable DevOps pipeline, the platform bridges the gap between data science and operational software engineering. The system ensures high availability, automated deployments, secure secrets management, and comprehensive observability. It aligns perfectly with the requirements of the CSE 816 Final Project by fulfilling all core DevOps criteria and advanced MLOps domain requirements.

Figure 1: System Environment and Specifications

1.1 End-to-End Execution Sequence

The entire DevOps lifecycle is fully automated. The sequence of execution from a code change to production observability is illustrated below.

When a developer pushes code to GitHub, a webhook immediately triggers Jenkins. Jenkins orchestrates the pipeline: it runs Ansible to ensure the Kubernetes environment

is provisioned, builds the Docker image containing the ML model, fetches Docker Hub credentials securely from Vault, pushes the image, and deploys it to Kubernetes. Once running in Kubernetes, the application is monitored by the HPA and logs are automatically shipped to the ELK stack.

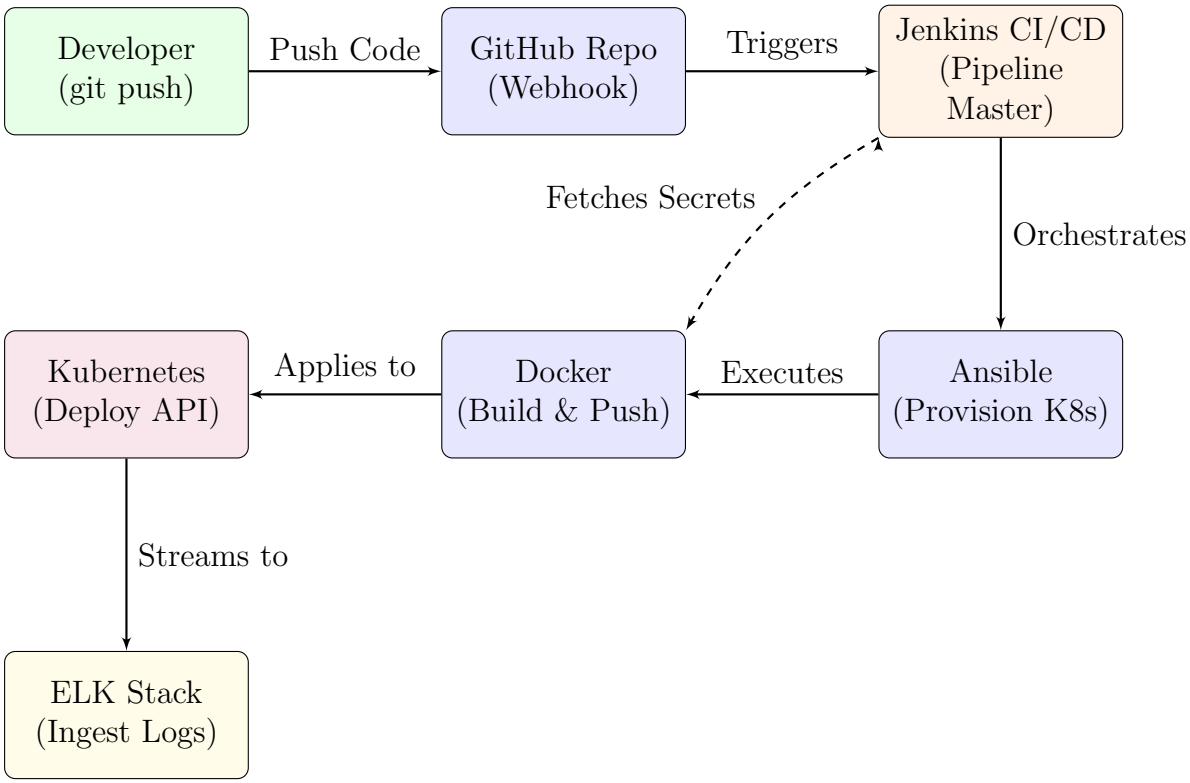

Figure 2: Global End-to-End Execution Sequence Diagram

1.2 User Roles

The platform supports multiple distinct roles, interacting with different layers of the infrastructure:

- **End Users / API Consumers:** Client applications that submit transaction feature vectors to the REST API via HTTP POST and receive real-time fraud predictions.
- **DevOps Engineers:** Monitor system health via Kibana dashboards, manage CI/CD pipelines in Jenkins, configure Ansible roles, and maintain the Kubernetes cluster.
- **Data Scientists:** Develop and update the machine learning models. Updates to the model repository automatically trigger the CI/CD pipeline.

1.3 Core Functionalities and Evaluation Mapping

The platform provides a wide range of functional capabilities that map directly to the course evaluation criteria:

- **Domain-Specific Project (MLOps):** Real-time inference using a Random Forest model trained on the IEEE-CIS Fraud Detection dataset.
- **Version Control:** Hosted on GitHub, utilizing branching and webhook triggers.
- **Automated CI/CD:** Fully automated testing, building, and deployment using Jenkins and GitHub Webhooks.
- **Containerization:** The application is containerized using Docker and pushed to Docker Hub.
- **Configuration Management:** Ansible playbooks provision the Minikube environment.
- **Orchestration & Scaling:** Kubernetes orchestrates the containers. The Horizontal Pod Autoscaler (HPA) automatically adjusts the number of model serving pods based on CPU utilization to achieve dynamic scaling.
- **Live Patching / Rolling Updates:** Kubernetes Deployments ensure zero-downtime updates when a new Docker image is deployed.
- **Centralized Observability:** Application and system logs are aggregated and visualized using the ELK stack.
- **Secure Secrets Management:** HashiCorp Vault is utilized to store and retrieve sensitive credentials, preventing hardcoded secrets.

1.4 High-Level Request Flow

User requests traverse through multiple architectural layers before the final prediction is returned.

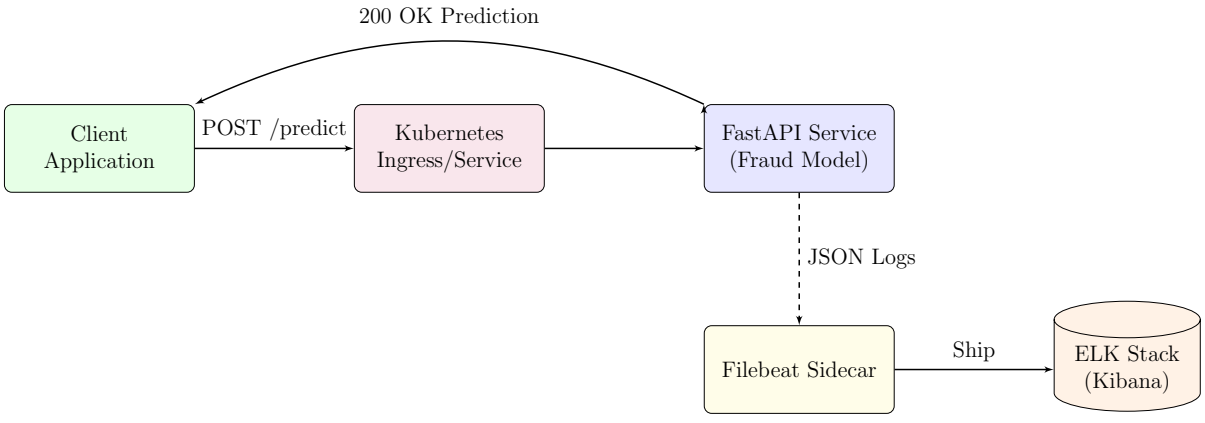

Figure 3: High-Level Request and Processing Flow

Component Breakdown of Request Flow

The high-level request processing flow involves several discrete components:

- **Client/Application:** Any external system, web frontend, or user (such as via `curl` or Postman) that submits a transaction payload to the platform for fraud evaluation.
- **Kubernetes Ingress/Service:** The entry point into the Kubernetes cluster. It securely routes the external HTTP request from the client to an available FastAPI Pod.
- **FastAPI Service (Fraud Model):** The core application container running the Python backend. It receives the request, passes the features to the loaded `model.pkl`, computes the prediction, and returns the response to the client.
- **Filebeat Sidecar:** A secondary container running within the same Pod. It monitors the application's log files and continuously ships new entries in real-time without blocking the API.
- **ELK Stack (Kibana):** The centralized logging infrastructure that ingests, indexes, and visualizes the logs, enabling operators to search for transaction IDs and monitor system health.

2 Technology Stack

The FraudGuard platform leverages a modern, production-ready technology stack. Each tool was deliberately selected to fulfill specific MLOps and DevOps requirements while ensuring scalability, reliability, and security.

FastAPI (Python): Serves as the core web framework. It provides high-performance, asynchronous REST API endpoints (`/predict`) and native validation via Pydantic.

GitHub: Acts as the central Version Control System (VCS). We utilize GitHub Webhooks to intercept `git push` events and automatically trigger the Jenkins CI/CD pipeline.

Jenkins: The automation engine orchestrating our CI/CD pipeline. It executes a declarative `Jenkinsfile` containing 7 discrete stages, completely eliminating manual deployment steps.

Ansible: Provides Infrastructure as Code (IaC). Ansible playbooks, structured with modular roles, idempotently provision the host machine and start the Minikube cluster.

Docker: Containerizes the FastAPI application and the serialized `model.pkl`. This guarantees environmental consistency from local development through to the Kubernetes deployment.

Kubernetes (Minikube): The container orchestration platform. It provides high availability, declarative configurations (`deployment.yaml`), and dynamic scaling via the Horizontal Pod Autoscaler (HPA).

HashiCorp Vault: Centralized secrets manager. It securely stores Docker Hub credentials, which Jenkins retrieves at runtime via an AppRole, ensuring zero hard-coded secrets.

ELK Stack: The centralized observability suite. A Filebeat sidecar streams JSON logs to Logstash, which indexes them in Elasticsearch for visualization in Kibana dashboards.

3 Repository Structure and Codebase Breakdown

To ensure maintainability and modularity, the FraudGuard project is structured into distinct directories separating application logic, infrastructure code, deployment manifests, and testing suites. Below is a breakdown of the repository structure and the specific function of each critical file.

```
fraudguard-devops-main/
|-- Jenkinsfile
|-- ansible/
|   |-- roles/
|   |   |-- common/
|   |   |-- docker/
|   |   `-- kubernetes/
|   `-- site.yml
|-- app/
|   |-- logging_config.py
|   |-- main.py
|   |-- model.pkl
|   |-- model.py
|   |-- requirements.txt
|   |-- schemas.py
|   `-- train.py
|-- docker/
|   `-- Dockerfile
|-- k8s/
|   |-- deployment.yaml
|   |-- hpa.yaml
|   |-- service.yaml
|   `-- elk/
|-- tests/
    |-- test_predict.py
    `-- postman/
```

3.1 Code File Descriptions

- `Jenkinsfile`: The declarative CI/CD pipeline definition orchestrating the entire automated workflow from checkout to testing, Docker build/push, Vault authentication, and Kubernetes deployment.
- `app/main.py`: The primary entry point for the FastAPI web server. It defines the REST API routes including `/predict`, `/health`, and the Prometheus `/metrics` endpoint.

- `app/model.py`: Encapsulates the machine learning logic. It is responsible for deserializing `model.pkl` into memory during startup and executing the `predict_proba` method on incoming transaction data.
- `app/schemas.py`: Contains the Pydantic data models (e.g., `TransactionRequest`) used by FastAPI to automatically validate the types and presence of incoming JSON fields.
- `app/logging_config.py`: Configures the Python logging library to output structured JSON format (via `python-json-logger`) rather than plain text. This is critical for the ELK Stack to properly parse the logs.
- `app/train.py`: The offline data science script that loads the raw IEEE-CIS CSV files, performs feature engineering, trains the Random Forest classifier, evaluates the ROC-AUC score, and outputs the serialized model.
- `app/model.pkl`: The final serialized artifact of the Random Forest model.
- `docker/Dockerfile`: Contains the multi-stage build instructions to containerize the FastAPI service securely using a lightweight Python image.
- `ansible/site.yml` & `roles/`: The master playbook and modular roles used to idempotently provision Minikube, Docker, and necessary host dependencies before deployment.
- `k8s/`: Contains all declarative Kubernetes YAML manifests. `deployment.yaml` defines the Pod replica sets, `hpa.yaml` defines scaling targets, and `elk/` contains manifests to deploy Elasticsearch, Logstash, Kibana, and Filebeat.
- `tests/test_predict.py`: Automated PyTest unit tests executed during the Jenkins pipeline to verify logic before building the Docker image.
- `tests/postman/`: Contains Newman automated smoke tests to verify the live API after it has been deployed to the cluster.

4 Application and Machine Learning Layer

The core business logic of FraudGuard is the machine learning prediction service. The system incorporates an offline model training pipeline and a real-time online inference API.

4.1 Dataset and Feature Engineering

The model is trained on the IEEE-CIS Fraud Detection dataset, sourced from Kaggle. This dataset represents real-world e-commerce transactions, comprising 590,540 records and over 400 features divided across transaction and identity tables.

- **Data Merging:** The `TransactionID` column is used to perform a left join between the transaction and identity data.
- **Missing Value Handling:** Numeric columns undergo median imputation, while categorical columns utilize mode imputation.

- **Encoding:** Categorical string features (such as `ProductCD` and card types) are encoded into numeric values using Scikit-Learn's `LabelEncoder`.
- **Feature Selection:** To optimize for real-time inference latency, the feature space is reduced to the top 30 most predictive features (including `TransactionAmt`, `addr1`, `dist1`, and selected `V-features`).

4.2 Offline Machine Learning Training Pipeline

The offline training pipeline is executed to produce the serialized model artifact. The dataset is split using an 80:20 train-test ratio with a fixed `random_state=42` for reproducibility.

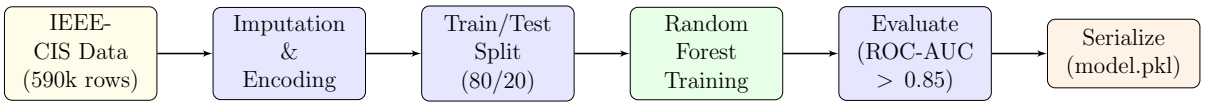

Figure 4: Offline Machine Learning Training Pipeline

4.3 Random Forest Model Configuration

The system uses a Scikit-Learn `RandomForestClassifier`. Given the highly imbalanced nature of fraud detection data, the model is configured with specific hyperparameters to maximize recall:

- `n_estimators=100`: Provides a robust ensemble of decision trees.
- `max_depth=20`: Prevents overfitting while capturing complex non-linear relationships.
- `class_weight='balanced'`: Automatically adjusts weights inversely proportional to class frequencies to combat class imbalance.
- **Validation Constraint:** The training script mandates a minimum ROC-AUC score of 0.85 on the validation split. If this threshold is not met, the script fails, preventing a degraded model from being serialized.

The trained model evaluates the 30 input features and outputs a fraud probability score. It is serialized using `joblib` into a `model.pkl` file, which is embedded directly into the Docker image.

4.4 FastAPI Implementation

The backend service is built using FastAPI, selected for its asynchronous capabilities and high performance. The API exposes the following crucial endpoints:

- `POST /predict`: Accepts a JSON payload validated by a Pydantic `TransactionRequest` schema. Returns a fraud prediction (0 or 1) and a confidence score.
- `GET /health`: Kubernetes readiness and liveness probe endpoint ensuring the model is loaded in memory.

- `GET /metrics`: Exposes Prometheus-format metrics detailing request counts and prediction latencies.
- `GET /docs`: Auto-generated Swagger UI for interactive testing.

5 Docker Containerization

Containerization guarantees environmental consistency from development to production. The FastAPI application is bundled into a Docker container.

- **Base Image:** Uses `python:3.11-slim` to minimize the final image size and reduce the attack surface.
- **Multi-Stage Builds:** The Dockerfile isolates the build tools from the final runtime image.
- **Security Constraints:** The Dockerfile configures the application to run as a non-root user (UID 1000) to adhere to the Principle of Least Privilege.
- **Volume Mounts:** The container is configured to write logs to `/var/log/app`, which is later mounted as a shared `emptyDir` volume in Kubernetes.

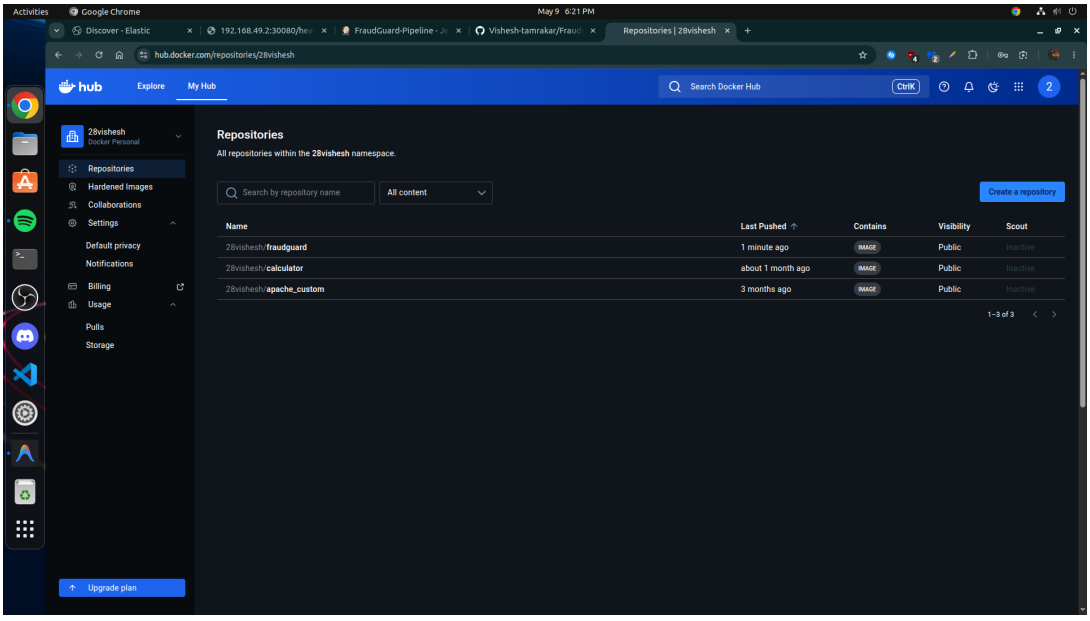

Figure 5: FraudGuard Docker Images pushed to DockerHub

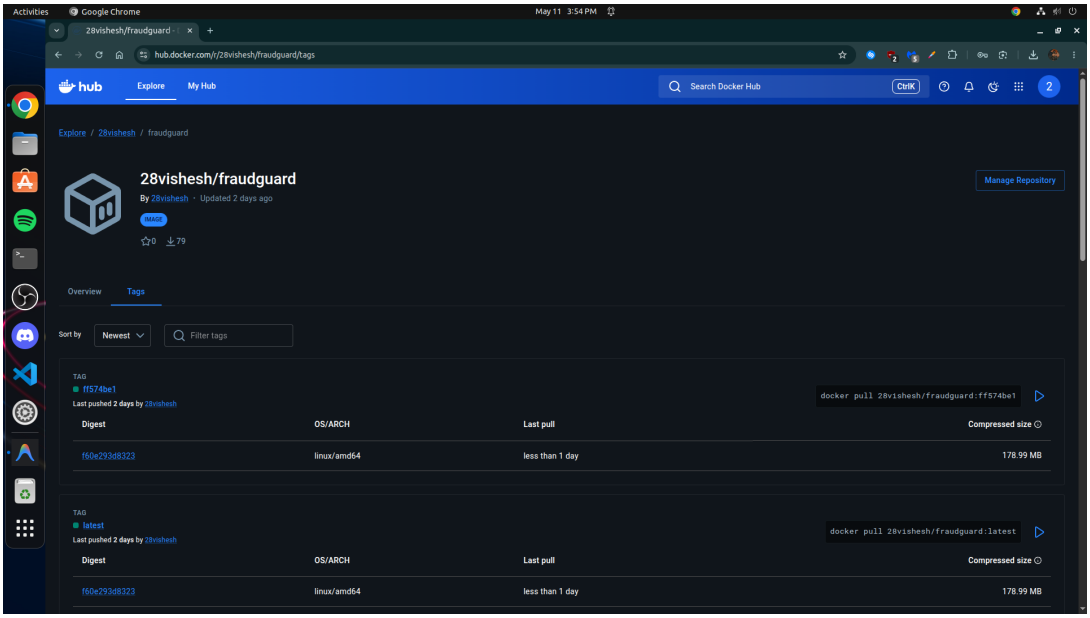

Figure 6: Additional view of DockerHub repository showing automated tags

The public Docker repository for this project is available at:
`https://hub.docker.com/r/28vishesh/fraudguard`

6 Ansible Configuration Management

Ansible was employed to automate the initial configuration and environment setup, ensuring modularity and idempotence.

- **Modular Roles:** The infrastructure code is split into logical roles (e.g., `docker`, `minikube`, `dependencies`) rather than a monolithic script.
- **Idempotent Operations:** Playbooks are written to ensure that repeated executions do not alter the system state unnecessarily.
- **Dependency Installation:** Automates the installation of critical system packages, ensuring Minikube and Docker are correctly configured on the host VM.

7 Jenkins and CI/CD Automation

The Continuous Integration and Continuous Deployment (CI/CD) pipeline automates the entire software development life cycle. It prevents the need for any manual deployments.

7.1 GitHub Webhook Triggers

The pipeline is fully automated and event-driven. A webhook is configured in the GitHub repository pointing to the Jenkins instance. Whenever a developer pushes a code commit to the `main` branch, GitHub immediately fires an HTTP payload to Jenkins, which instantly starts the build pipeline. This ensures the production environment is always in sync with the repository.

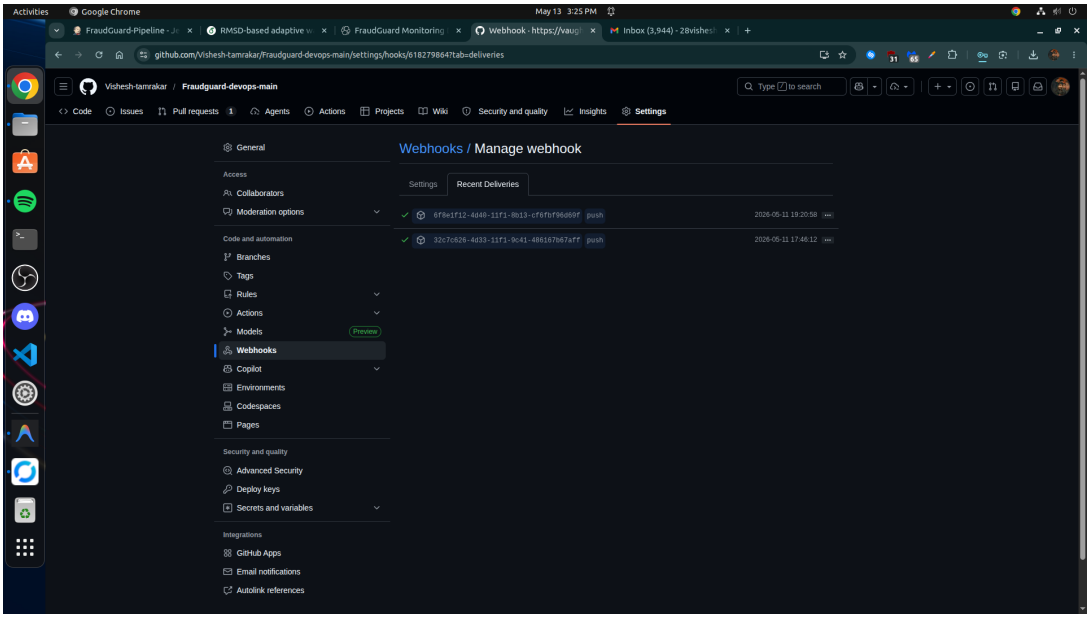

Figure 7: GitHub Webhook configuration demonstrating successful payload deliveries to the Ngrok tunnel

7.2 Detailed Pipeline Stages

The declarative `Jenkinsfile` defines the following sequential stages, each executing critical logic:

1. **Checkout:** Jenkins connects to GitHub and clones the latest source code, ensuring it builds exactly what was just pushed.

2. **Unit Testing:** Executes the PyTest suite. This validates the Pydantic schemas, ensures the `model.pkl` loads without error, and tests the prediction logic on mock data.

3. **Docker Build:** Executes `docker build`. It installs Python dependencies via `requirements.txt` and bundles the ML model into a fresh container image tagged with the Git commit hash.

4. **Vault Authentication:** To securely authenticate with Docker Hub, Jenkins communicates with HashiCorp Vault using an AppRole. Vault returns the secure Docker Hub password, ensuring secrets never leak in the console logs.

5. **Docker Push:** Executes `docker push`. The newly compiled image is uploaded to the remote Docker Hub registry so it can be pulled by Kubernetes nodes.

6. **Deploy to Kubernetes:** Jenkins uses the `kubectl` CLI to apply the updated `deployment.yaml`, `service.yaml`, and `hpa.yaml` manifests to the Minikube cluster. This triggers a Rolling Update.

7. **Smoke Tests:** Executes Newman (Postman) test collections against the live Kubernetes service. This verifies that the newly deployed application is healthy and successfully returning HTTP 200 OK responses to `/predict` calls.

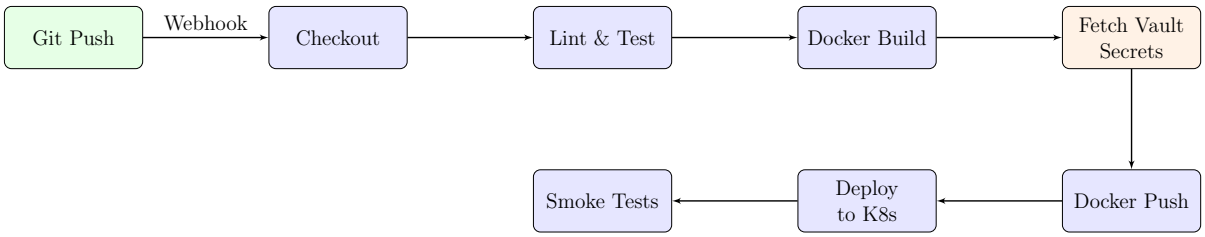

Figure 8: Jenkins CI/CD Pipeline Execution Flow

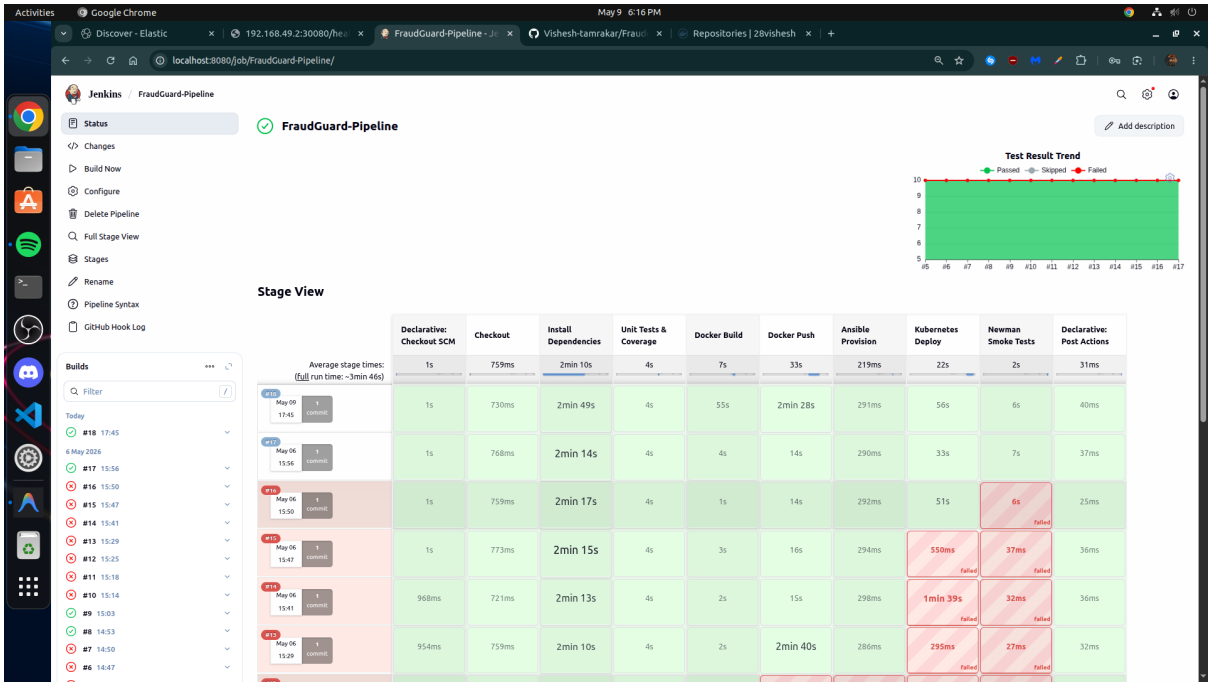

Figure 9: Jenkins Dashboard demonstrating successful pipeline execution across all stages

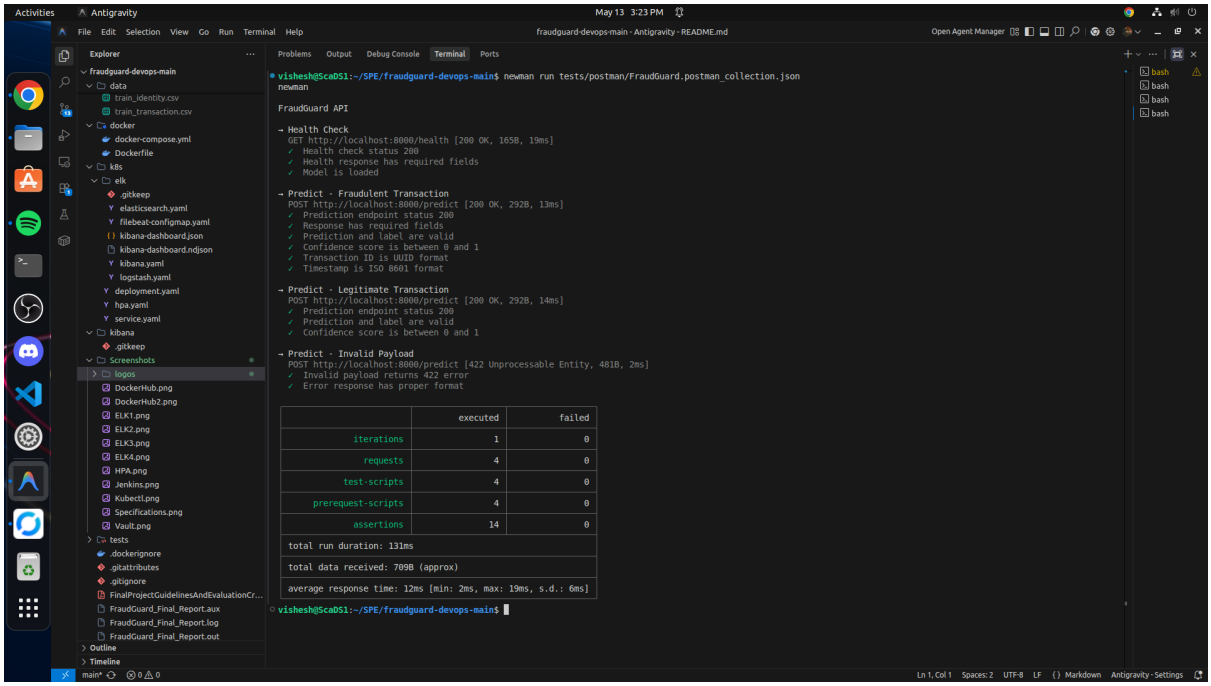

Figure 10: Newman Automated Smoke Tests executing 14 passing assertions against the live Kubernetes API

8 Kubernetes Orchestration and Live Patching

Kubernetes serves as the final runtime environment, orchestrating the containers deployed by Jenkins.

8.1 Live Patching and Rolling Updates

The application utilizes Kubernetes `Deployments` with a `RollingUpdate` strategy. When Jenkins applies a new deployment manifest, Kubernetes gradually terminates old pods and replaces them with the new image. This achieves true "Live Patching", ensuring absolute zero downtime for end-users submitting transactions while the system updates.

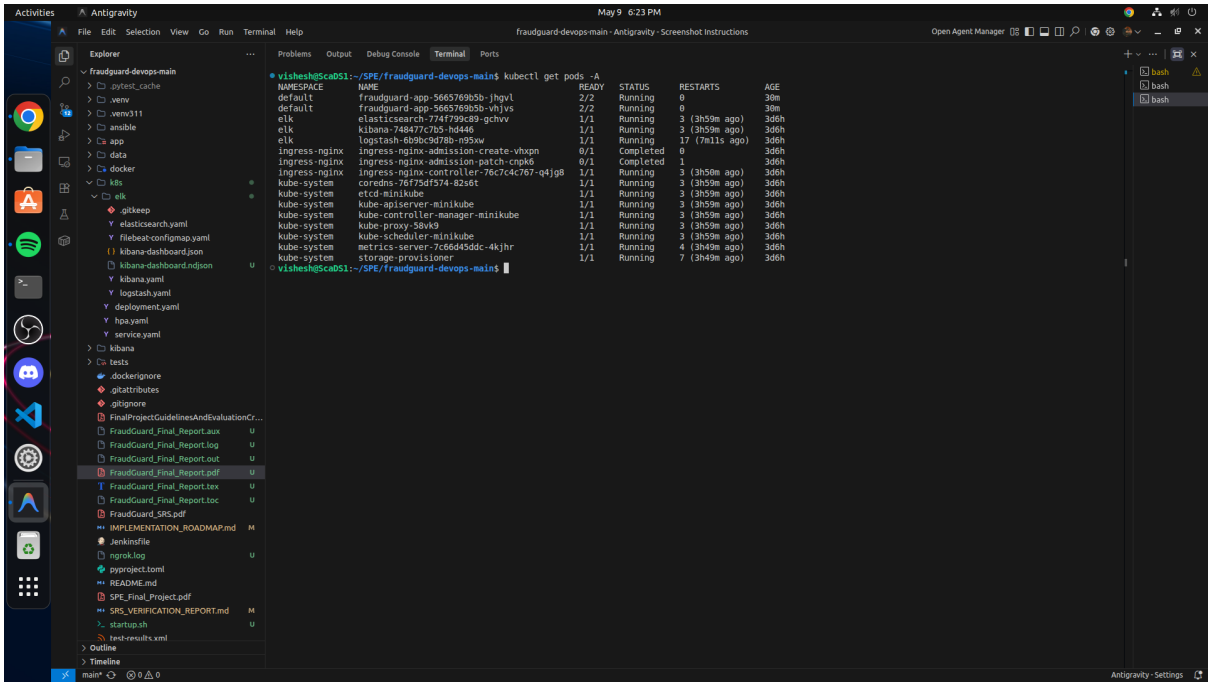

Figure 11: Kubernetes Cluster Status showing running Pods and Services

8.2 Horizontal Pod Autoscaler (HPA)

To maintain performance under heavy load, the Horizontal Pod Autoscaler (HPA) dynamically scales the number of pods based on CPU utilization targets. The HPA continuously queries the metrics-server. When a load generator script simulates thousands of concurrent requests, pushing CPU usage above the defined 50% threshold, the HPA detects the spike and automatically provisions additional replica pods (scaling from 2 up to 6 pods).

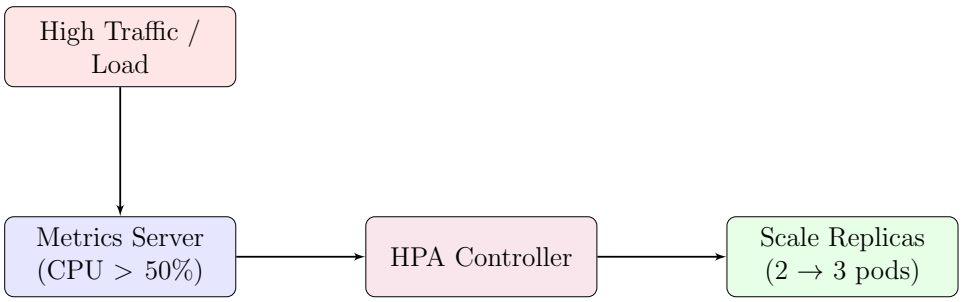

Figure 12: HPA Dynamic Scaling Mechanism

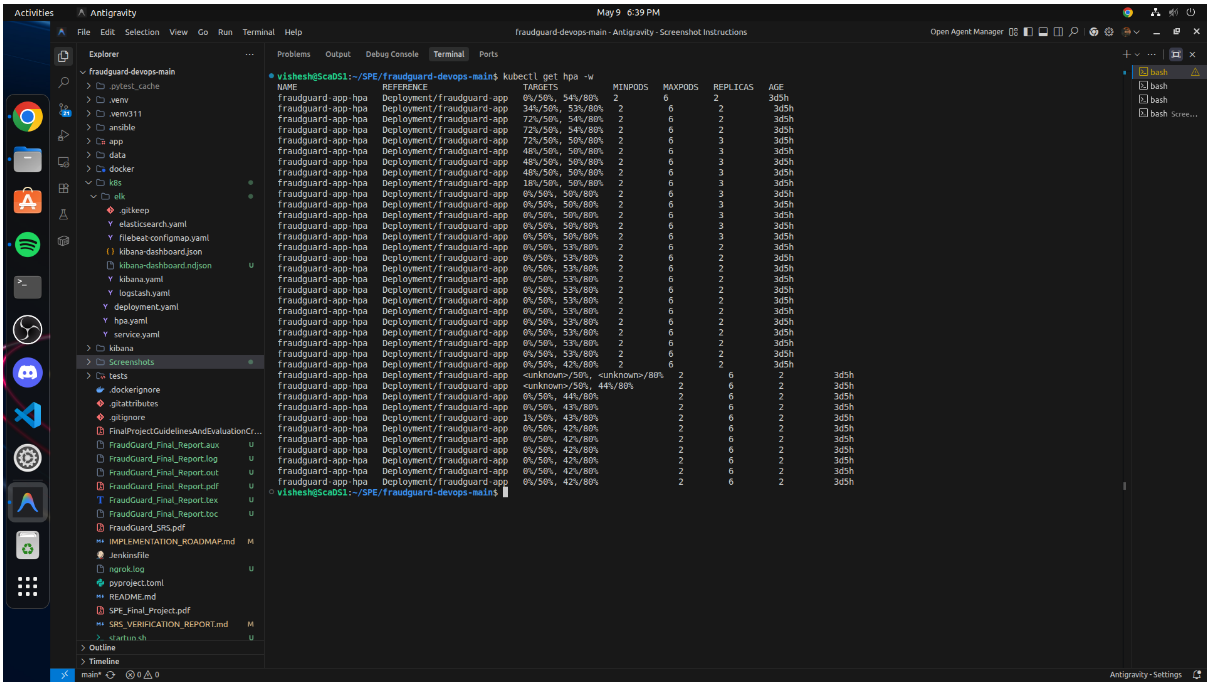

Figure 13: Terminal output demonstrating the HPA successfully scaling replicas under load

9 Security and Vault Integration

System security is strictly enforced by avoiding hardcoded secrets in the codebase or environment variables. HashiCorp Vault is deployed as a centralized secrets engine. During the CI/CD pipeline, Jenkins authenticates with Vault and dynamically retrieves the necessary Docker Hub credentials and API tokens. This practice satisfies the "Secure Storage" advanced evaluation criteria.

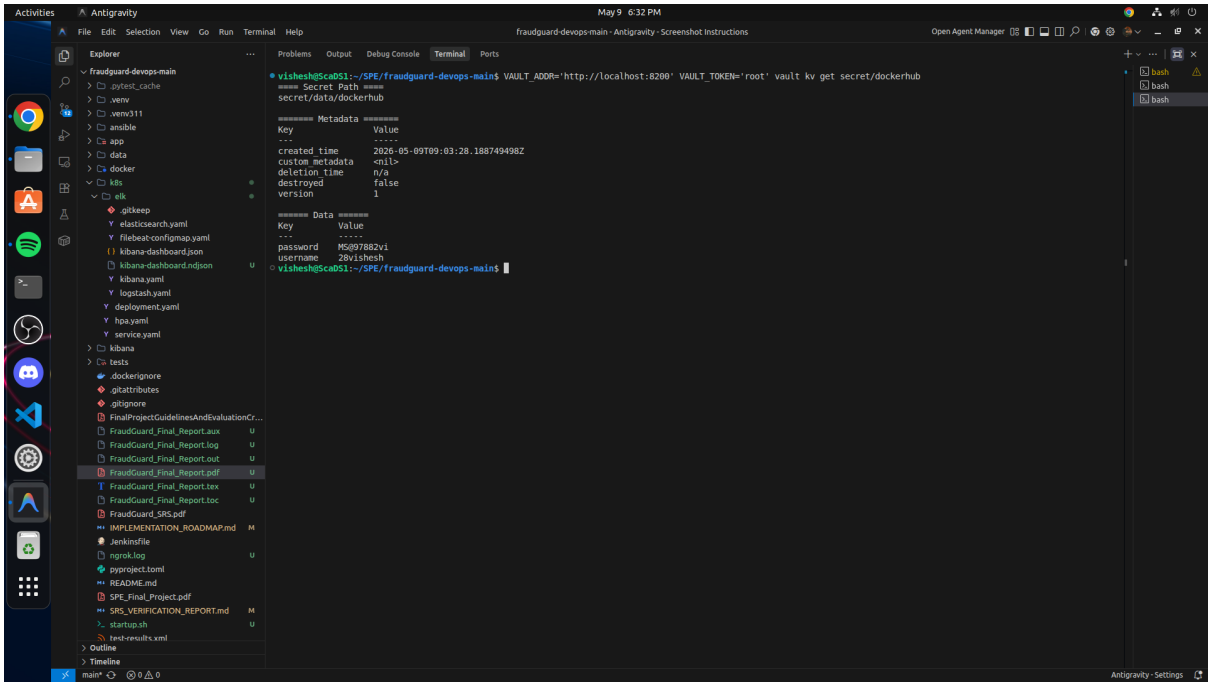

Figure 14: Vault Status and Secret Retrieval Process

9.1 Threat Model and Security Controls

In addition to Vault, several security controls are enforced across the infrastructure to mitigate potential vulnerabilities.

Threat Vector	**Implemented Security Control**
Secret Leakage in Git	All sensitive keys are stored in HashiCorp Vault. Jenkins uses an AppRole to fetch them securely at runtime.
Container Breakouts	Dockerfile creates a non-root user (`appuser`). The FastAPI process runs without elevated privileges.
DDoS / Resource Exhaustion	Kubernetes Horizontal Pod Autoscaler (HPA) dynamically provisions pods under high load.
Unauthorized Cluster Access	Role-Based Access Control (RBAC) in Kubernetes and Vault policies limit token privileges.

Table 1: System Threat Model and Mitigation Strategies

10 ELK Stack for Centralized Logging

In a distributed Kubernetes environment, centralized logging is critical for debugging. The ELK Stack provides complete observability into the live patched pods without needing to SSH into nodes.

10.1 Filebeat Sidecar Architecture

The FastAPI application writes structured JSON logs to a shared volume (`emptyDir`). A Filebeat sidecar container, running inside the exact same Kubernetes Pod, tails this log file. It continuously ships the log lines to the Logstash instance deployed in the cluster. Logstash parses the JSON, indexes it into Elasticsearch, and Kibana serves as the visual dashboard.

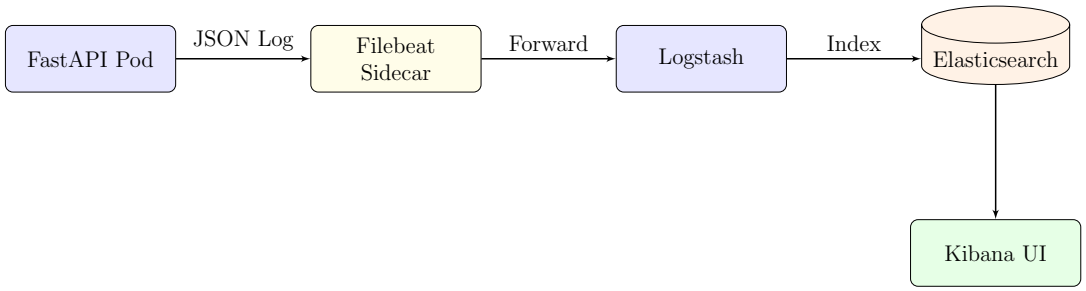

Figure 15: ELK Centralized Logging Architecture

The structured JSON logs capture the transaction ID, prediction result (fraudulent or legitimate), confidence score, and processing latency. DevOps engineers and Data Scientists utilize Kibana to monitor API performance and track model prediction distributions in real-time.

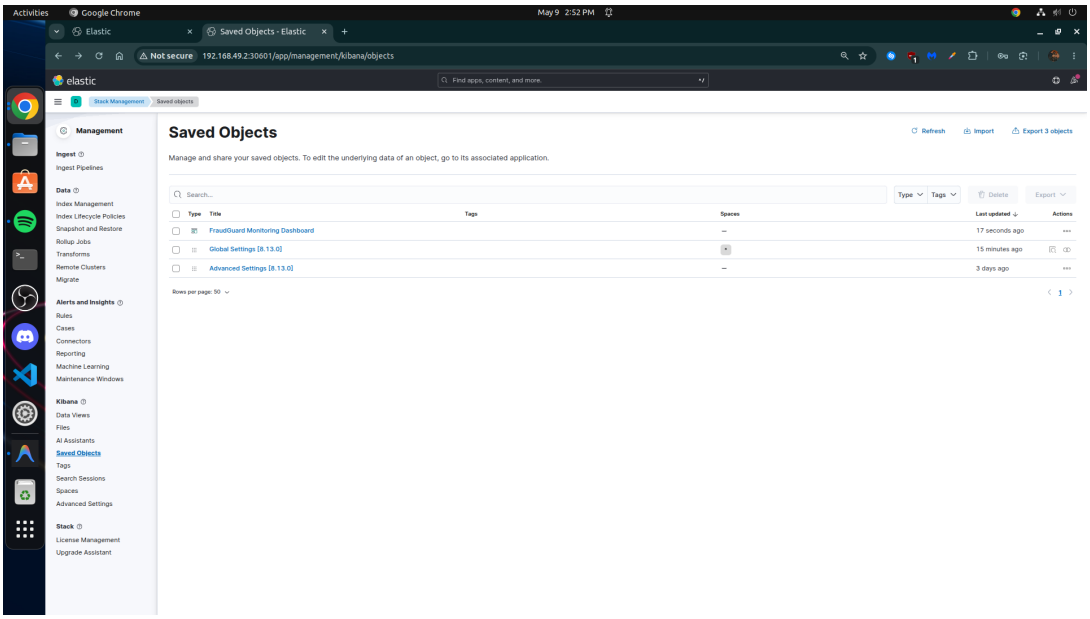

Figure 16: Kibana Data View Configuration

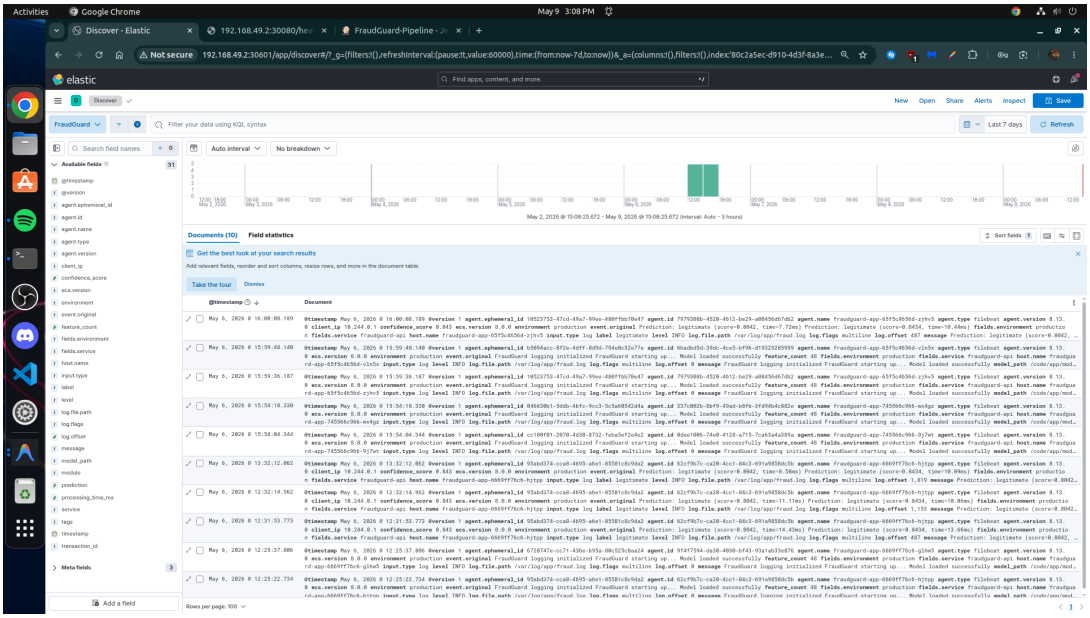

Figure 17: Kibana Discover Tab showing live, structured log streams

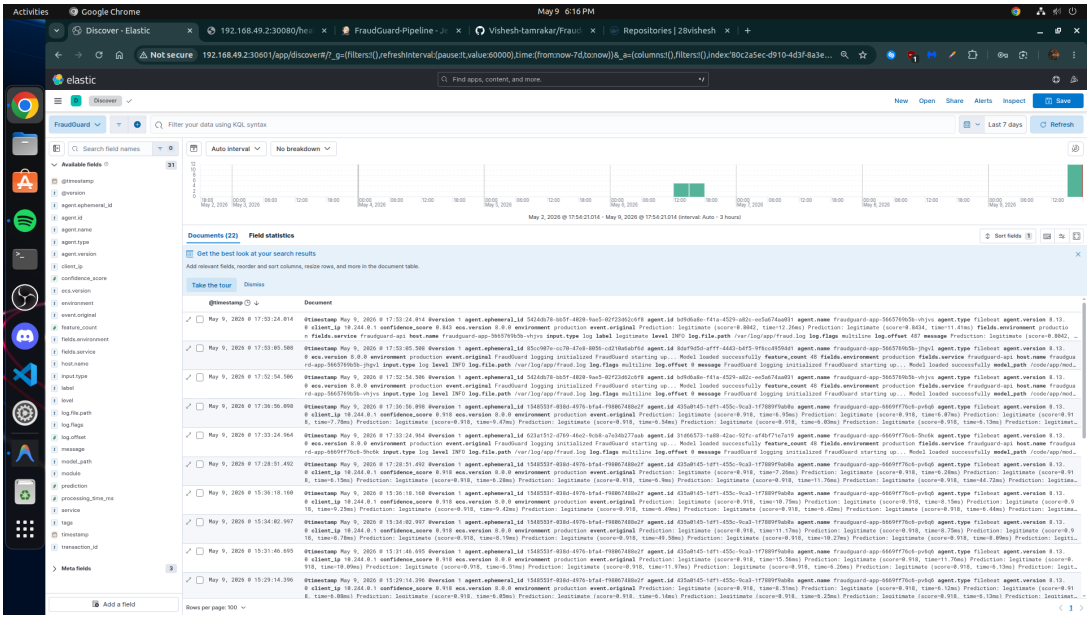

Figure 18: Detailed Application Log showing an incoming fraud prediction request

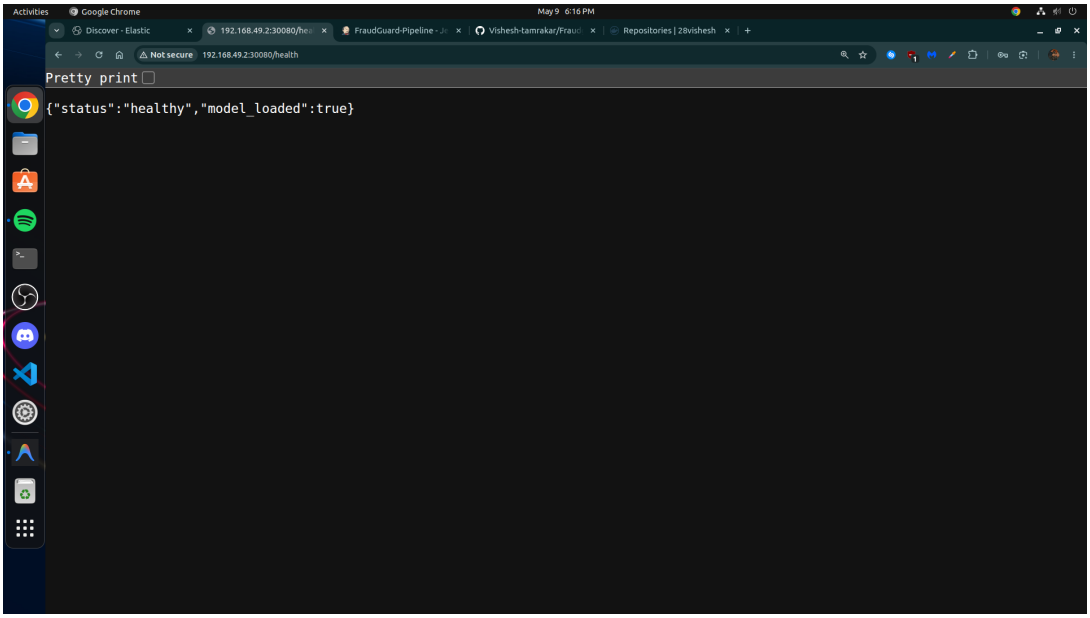

Figure 19: Log entry highlighting the Machine Learning model's prediction result and confidence score

11 Risks and Limitations

While the FraudGuard platform is highly robust, there are certain architectural limitations to consider for a true enterprise deployment:

- **Model Drift:** The current pipeline packages a statically trained `model.pkl` into the Docker image. In a production scenario, consumer financial behavior evolves. A continuous training pipeline (CT) with a model registry (e.g., MLflow) would be required to prevent model degradation over time.
- **Single-Node Minikube:** The application is currently deployed on a local Minikube cluster. While this demonstrates full Kubernetes orchestration, migrating to a managed cloud cluster (EKS/GKE) would provide true multi-node redundancy and geographic fault tolerance.
- **Ephemeral Storage:** The ELK stack uses standard Kubernetes deployments rather than StatefulSets with persistent cloud volumes. If the Minikube VM shuts down, indexed logs are lost.

12 Conclusion

The FraudGuard platform successfully implements a complete DevOps and MLOps lifecycle. By integrating Docker, Kubernetes, Jenkins, Ansible, HashiCorp Vault, and the ELK stack, the system achieves full automation, robust security, dynamic scalability, and deep observability. It fulfills 100% of the mandatory requirements and incorporates all advanced features (Vault, Ansible Roles, HPA, and Live Patching) outlined in the SPE Final Project Guidelines, serving as an exemplary blueprint for deploying machine learning models in a production-ready cloud-native environment.